

WorkRadar — Product Requirements Document

MVP Assignment

1. Problem Statement

- Engineering and product managers waste time checking multiple tools (Slack, Jira, GitHub, Email) to stay updated on team progress and blockers.
- There is no single, concise view to answer daily management questions quickly.

2. Target Users

Engineering Managers and Product Leads managing software development teams in fast-paced environments.

3. User Personas

Engineering Manager (“Priya”)

Manages 7 engineers, wants to spot blockers early and prioritize urgent team actions each morning.

Product Lead (“Sam”)

Oversees projects across 2 squads, needs quick insight into daily progress and any escalations requiring immediate response.

4. User Journey

- Manager logs onto WorkRadar web app in the morning.
- Lands on Dashboard with consolidated view: top work for the day, what happened today, what requires attention, who is blocked.
- Reviews AI-generated summary and scans the priority feed.
- Searches for a specific issue raised in a team meeting.
- Navigates to detail page, assigns it to a team member, snoozes a non-urgent task, and marks a completed item as done.
- Logs off with clear understanding of team status and action list.

5. Key Pain Points

- Context-switching between multiple tools is inefficient.
- Important updates and blockers are missed due to tool silos.
- Managers spend time compiling manual digests when insights should be automatic.

6. Success Metrics

- Manager can answer all four key questions within 3 minutes of login.
- 90% of sample priority actions are identified correctly in summary/feed.
- 100% of blockers are surfaced on the dashboard.
- Task actions (Assign/Mark Done/Snooze) update status instantly for demo.
- Positive user feedback: “Faster than checking Slack + Jira + email.”

7. MVP Scope

In Scope:

- Web-only mock app with sample (not live) data.
- Dashboard summarizing top work, recent updates, urgent items, blocked users.
- Static AI-generated daily summary (sample text).
- Priority feed (sortable, filterable), search, and item detail.
- Three core actions: Assign, Mark Done, Snooze (update status locally only).

Out of Scope:

- Real backend integrations (Slack, Jira, GitHub, email APIs).
- Mobile support or responsive UI.
- Team switching or admin/multi-manager roles.
- Notification systems or real-time collaboration.

8. Assumptions

- MVP is built by a solo developer and is for demo/coursework use only.
- Users will be provided sample/mock data for demo/testing.
- AI summary is pre-written and not dynamically generated.
- No persistent database or authentication required for MVP demo.

9. Features Table

Must-Have	Nice-to-Have
Dashboard (consolidated, 4-question layout)	Sort/filter feed by user
AI-generated daily summary (static sample)	Export daily summary (PDF)
Priority feed (sample data, status icons)	Light/dark theme toggle
Search across all items	Customizable dashboard layout
Item detail page with activity log/status	Integration stubs (fake APIs)
Assign, Mark Done, Snooze actions (local only)	User profile customization

WorkRadar — AI Workflow Document

Course assignment · Solo developer: Nikhil

AI Tools Used

Tool	Role in the workflow
Claude	Strategy and prompt-engineering partner throughout. Drafted the PRD prompt, the 5 screen-design prompts, the MVP build prompts and design system, the mock dataset, and this document. Also used to inspect and QA the deployed app directly.
ChatPRD	Generated the Product Requirements Document from a structured prompt drafted in Claude.
Figma Make	Generated the 5 required high-fidelity screens (Dashboard, Priority Feed, Item Details, Search, Empty/Error) from screen-by-screen prompts.
Replit (Agent)	Built and hosts the functional MVP from a master prompt covering tech stack, pages, mock data, design system, and required actions.

Significant Prompts

Full prompts ran longer; representative excerpts below show the structure used at each stage.

PRD prompt (→ ChatPRD):

"WorkRadar is an AI-powered web app that gives engineering/product managers a single consolidated view of their team's work... Write a PRD with exactly these sections: Problem Statement, Target Users, Personas, User Journey, Pain Points, Success Metrics, MVP Scope, Assumptions, Features table (Must-Have vs Nice-to-Have)... entire PRD must fit on 2 pages."

Screen design prompt, e.g. Dashboard (→ Figma Make):

"Design a Dashboard screen for WorkRadar... Include: an AI-generated daily summary card, a four-panel layout – Top work for today / What happened today / What requires immediate attention / Who is blocked – and a left sidebar (Dashboard, Priority Feed, Search)."

MVP master scaffold prompt (→ Replit):

"Build WorkRadar... React + Tailwind, no backend/database/auth – mock-data MVP. Seed with an 18-item mock dataset [schema attached]. Pages: Dashboard, Priority Feed, Item Detail, Search, Empty/Error. Actions: Assign, Mark Done, Snooze update local shared state. Design system: [token block attached]."

Follow-up refinement prompt (→ Replit, after first pass):

"On the Dashboard, make sure the four panels are literally labeled 'Top work for today,' 'What happened today,' 'What requires immediate attention,' and 'Who is blocked' – and that each pulls from the shared mock data array, not separate hardcoded lists."

What AI Generated

- Full 9-section PRD draft: problem statement, personas, user journey, pain points, success metrics, MVP scope, assumptions, and a Must-Have/Nice-to-Have features table.

- All 5 required high-fidelity screens: Dashboard, Priority Feed, Item Details, Search, Empty/Error state.
- A complete design system: 8 named color tokens, a 3-typeface pairing (Space Grotesk / Inter / IBM Plex Mono), spacing and radius scale, and one signature motion element.
- An 18-item mock dataset spanning all 4 sources, 5 statuses, and 3 explicitly blocked items with blocker reasons.
- A fully functional MVP: 4-panel dashboard with AI summary, filterable/sortable priority feed, item detail with activity timeline, working search (empty and results states), and Assign/Mark Done/Snooze actions wired to shared state.

What I Modified

- Chose an explicit design-token system over screenshot-matching after determining AI builders apply written tokens more reliably than interpreting images across 5 separate screen generations.
- Curated the mock dataset beyond the AI's first pass to guarantee a real spread across sources/statuses/priorities, and tied assignee names back to the PRD's own personas (Priya, Sam) for narrative consistency in the demo.
- QA'd the deployed Replit app screen-by-screen against the PRD's required features directly in-browser; found that Priority Feed action buttons only appear after a click rather than being always-visible, and flagged it for a follow-up fix.
- Sent iterative follow-up prompts after the first build pass — enforcing exact PRD panel wording and lifting local state so status changes persist across pages — since the one-shot generation didn't fully match spec.

Key Trade-offs

- **Design tokens over screenshots** — more consistent styling across AI-generated screens, at the cost of only loose (not pixel-exact) fidelity to the original Figma mockups.
- **Static AI summary vs. a live LLM call** — faster and more reliable to demo, but doesn't showcase a dynamically generated summary in action (documented as an explicit PRD assumption).
- **Local-state-only actions, no backend** — matches MVP/course scope and avoids auth/infra work, but actions reset on page refresh and won't survive a demo restart.
- **One restrained animated flourish** (a radar-sweep loading indicator) instead of richer motion throughout — keeps the app feeling calm and intentional rather than templated, at some cost of visual richness.
- **Three separate AI tools instead of one integrated pipeline** — ChatPRD, Figma Make, and Replit were each faster for their own stage, at the cost of manually carrying context (prompts, tokens, data) between tools.